Lab 3: Component Modelling & Architectural Pattern Selection

Public Bus Live Tracking & Crowding Estimator

SRN: PES1UG24AM396

1. UML Component Diagram

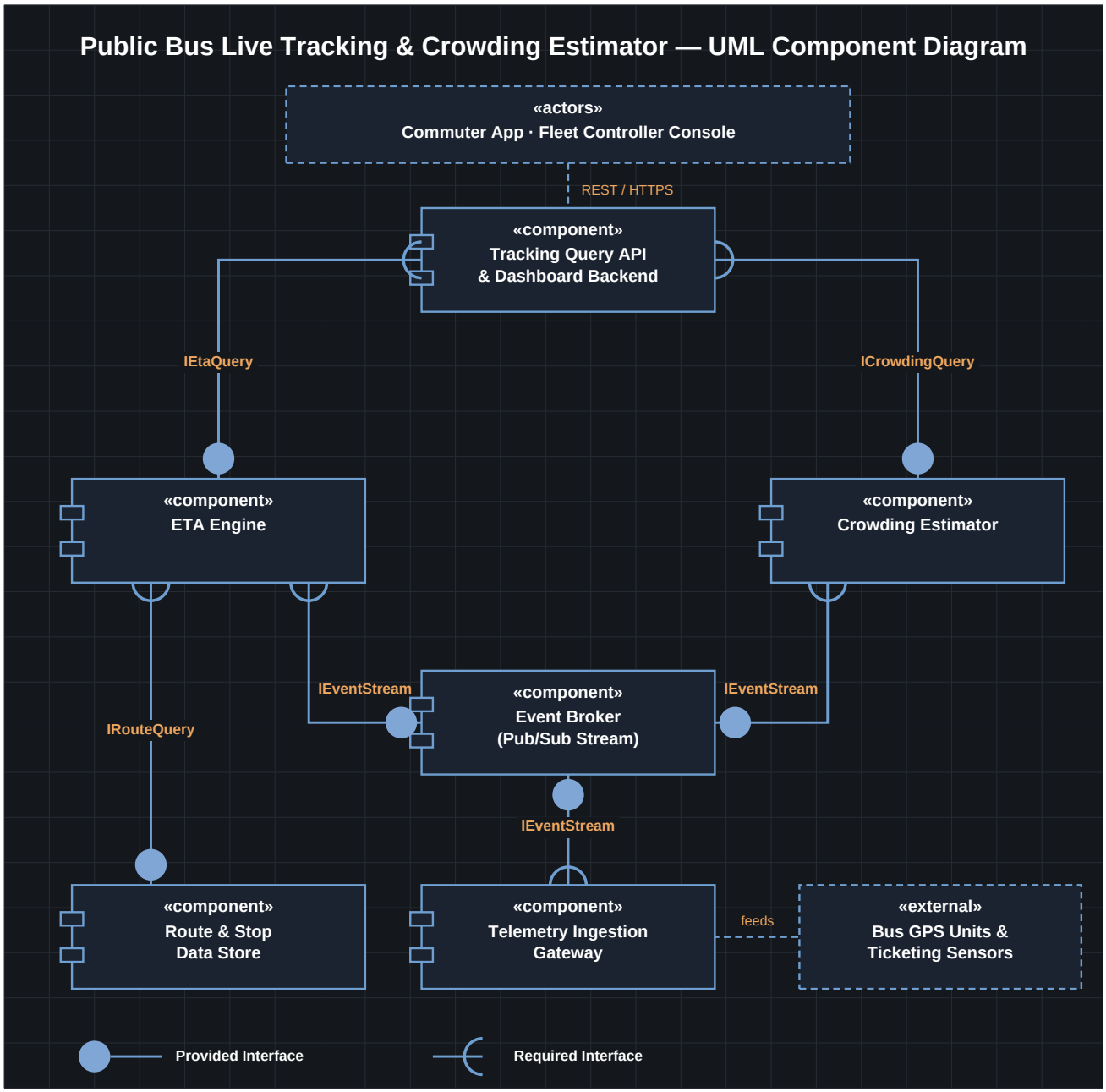

Figure 1 — Component diagram showing 6 components and 4 provided interfaces (6 required-interface dependencies).

2. Written Justification

Architecture Selection

We chose **Event-Driven (Publish–Subscribe) Architecture** for the Public Bus Live Tracking & Crowding Estimator.

Architectural Choice

Bus GPS positions and ticketing-sensor counts enter through the Telemetry Ingestion Gateway, which validates them and publishes them as events to the Event Broker (IEventStream). The ETA Engine and the Crowding Estimator subscribe to that stream independently: the ETA Engine recalculates arrival times using route geometry from the Route & Stop Data Store (IRouteQuery), and the Crowding Estimator computes occupancy levels per bus. Both publish their results to the Tracking Query API & Dashboard Backend (IEtaQuery, ICrowdingQuery), the only component the Commuter and Fleet Controller interfaces talk to.

Reason 1 — Matches Continuous, High-Volume Streaming Data

The platform must handle telemetry from up to 1,000 buses reporting every 5 seconds (about 200 position updates per second, plus ticketing events), and that data arrives whether or not anyone is asking for it. A request–response Layered or Client-Server design would force ingestion to wait on ETA and crowding computation, and would push the system to poll. With a broker in the middle, producers publish and move on, and consumers process events at their own pace, which is the natural shape of this workload.

Reason 2 — Loose Coupling, Fault Isolation and Extensibility

The ETA Engine and Crowding Estimator know only the event stream, not each other or the data sources. If the Crowding Estimator fails or is redeployed, ETAs keep updating. New capabilities, such as delay alerts for the Fleet Controller or historical analytics, can be added as extra subscribers without changing the Gateway or existing consumers. This is harder in a monolithic or tightly layered design, where such a change ripples through several modules.

Security Advantage

Only the Telemetry Ingestion Gateway accepts connections from buses and sensors, so it is the single place where each device is authenticated (e.g., mutual TLS or signed device tokens), and where payloads are validated and rejected if malformed or spoofed before reaching the stream. Internal components (Broker, ETA Engine, Crowding Estimator, Data Store) are never exposed to external traffic. Commuters and Fleet Controllers reach only the read-oriented Query API, which serves precomputed results and applies role-based access, so they never touch raw telemetry or internal services. Ticketing data is handled as aggregate passenger counts, not personal rider information.

Performance Benefit

The broker buffers traffic, absorbing peak-hour bursts without dropping updates or slowing buses. Events can be partitioned by bus or route, so ETA and crowding consumers scale horizontally as the fleet grows. Because results are computed when events arrive and stored in the Query API, a commuter request becomes a fast lookup rather than a fresh computation, which helps meet the near-real-time ETA refresh and 1,000-vehicle load targets from the requirements.